

CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

September 25, 2025

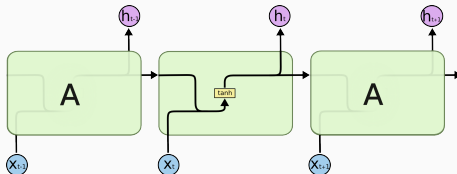
Lecture 10

Problems of Classical RNN

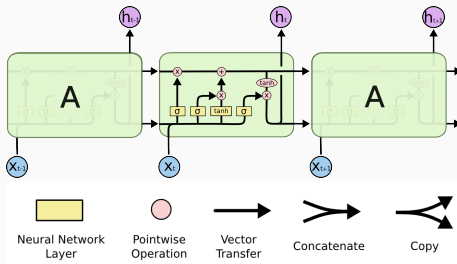
- Hard to capture long-term dependencies
- Hard to solve (vanishing gradient problem)
- Solution:
 - LSTM (Long Short Term Memory networks)
 - GRU (Gated Recurrent Unit)
 - ...

LSTM (<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>)

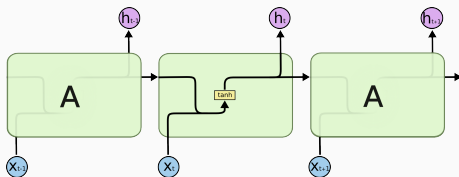
- RNN:



- LSTM:

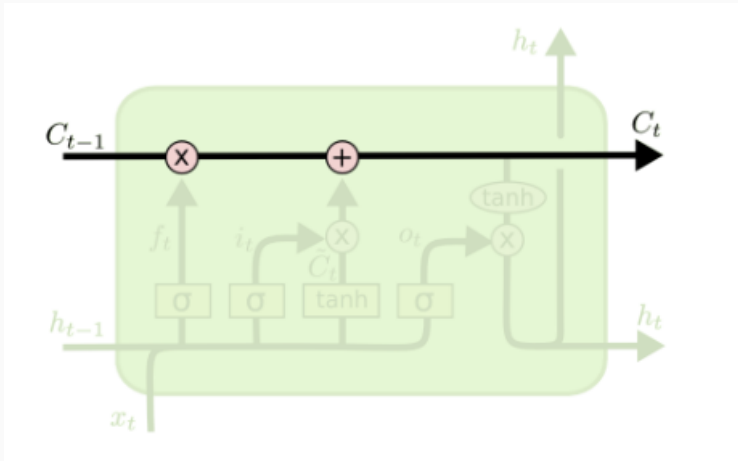


- RNN:



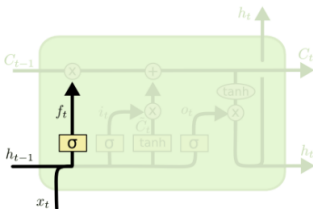
- $h_t = \tanh(W_h x_t + U_h h_{t-1} + b_h)$
- $y_t = \tanh(W_y h_t + b_y)$

LSTM



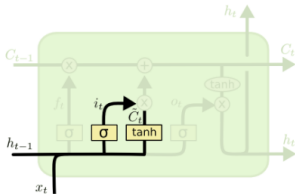
In addition to hidden states, add a more straightforward cell state that can be carefully controlled.

LSTM



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

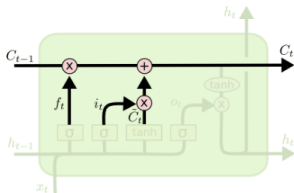
LSTM



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

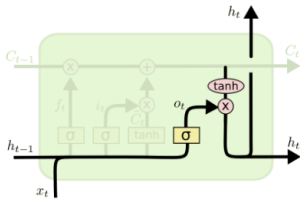
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

LSTM



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

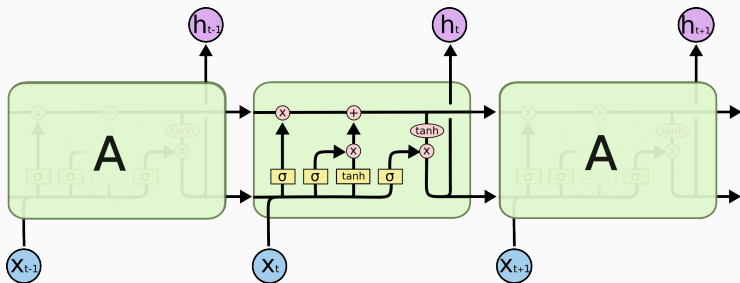
LSTM



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

LSTM



$$f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f)$$

$$i_t = \sigma_g(W_i x_t + U_i h_{t-1} + b_i)$$

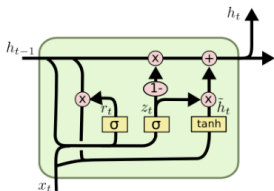
$$o_t = \sigma_g(W_o x_t + U_o h_{t-1} + b_o)$$

$$\tilde{c}_t = \sigma_c(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \sigma_h(c_t)$$

Gated Recurrent Unit



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

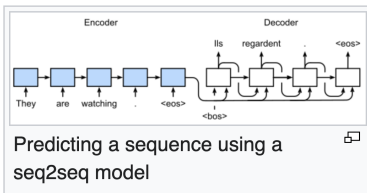
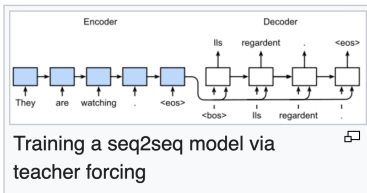
$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

It combines the forget and input gates into a single “update gate.”

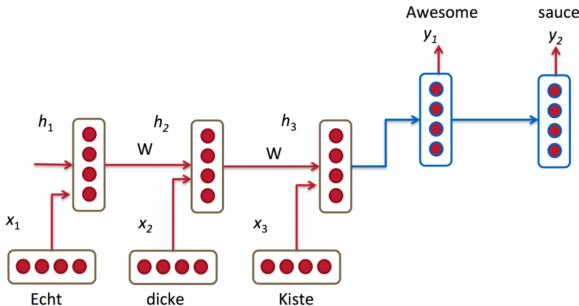
It also merges the cell state and hidden state, and makes some other changes.

Sequence to Sequence

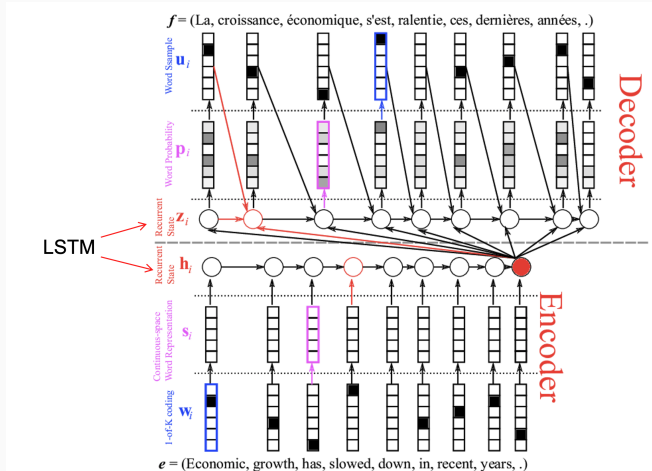


Neural Machine Translation (NMT)

- Produce the translated sentence from an input sentence
- Training data: a set of input-output pairs (supervised setting)
- Encoder-decoder approach:
 - Encoder: Use (RNN/LSTM) to encode the input sentence into a latent vector
 - Decoder: Use (RNN/LSTM) to generate a sentence based on the latent vector



Neural Machine Translation



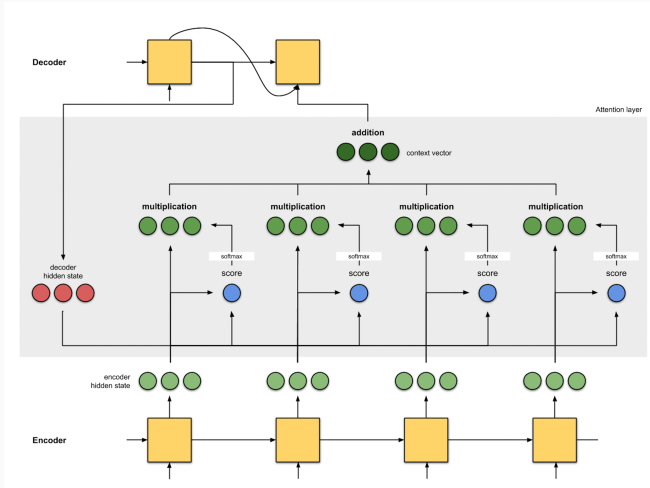
Attention in NMT

- Usually, each output word is only related to a subset of input words
(e.g., for machine translation)
- Let $\mathbf{u}$ be the **current decoder latent state**
 $\mathbf{v}_1, \dots, \mathbf{v}_n$ be the **latent sate for each input word**
- Compute the weight of each state by

$$\mathbf{p} = \text{Softmax}(\mathbf{u}^T \mathbf{v}_1, \dots, \mathbf{u}^T \mathbf{v}_n)$$

- Compute the context vector by $V\mathbf{p} = p_1 \mathbf{v}_1 + \dots + p_n \mathbf{v}_n$

Attention in NMT



(Figure from <https://towardsdatascience.com/neural-machine-translation-nmt-with-attention-mechanism>)

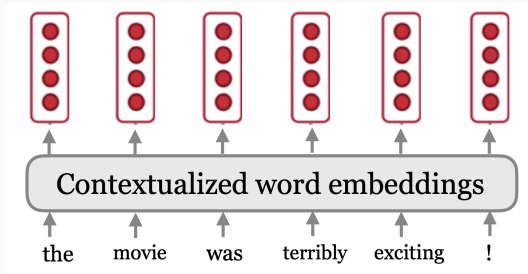
Contextual world representation

- The semantic meaning of a word should depend on its context

$[0.9, -0.2, 1.6, \dots]$
↑
open a bank account

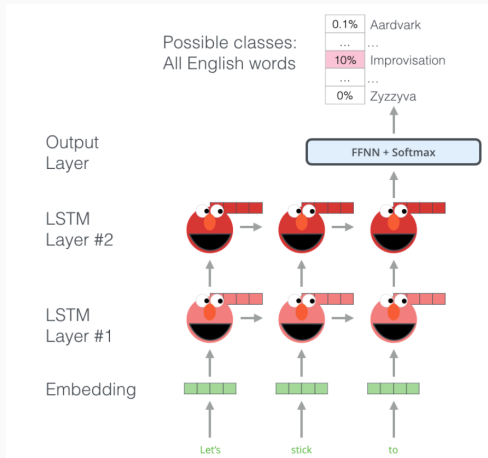
$[-1.9, -0.4, 0.1, \dots]$
↑
on the river bank

- Solution: Train a model to extract contextual representations on text corpus



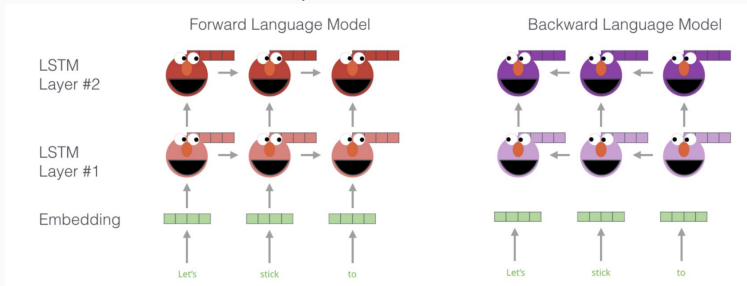
Language model pretraining task

- Predict the next word given the prefix
- Can be defined on any unlabeled document



ELMo (Peter et al., 2018)

- Key ideas:
 - Train a forward and backward LSTM language model on large corpus
 - Use the hidden states for each token to compute a vector representation of each word
 - Replace the word embedding by Elmo's embedding (with fixed Elmo's LSTM weights)



ELMo results

TASK	PREVIOUS SOTA		OUR BASELINE	ELMo + BASELINE	INCREASE (ABSOLUTE/ RELATIVE)
SQuAD	Liu et al. (2017)	84.4	81.1	85.8	4.7 / 24.9%
SNLI	Chen et al. (2017)	88.6	88.0	88.7 ± 0.17	0.7 / 5.8%
SRL	He et al. (2017)	81.7	81.4	84.6	3.2 / 17.2%
Coref	Lee et al. (2017)	67.2	67.2	70.4	3.2 / 9.8%
NER	Peters et al. (2017)	91.93 ± 0.19	90.15	92.22 ± 0.10	2.06 / 21%
SST-5	McCann et al. (2017)	53.7	51.4	54.7 ± 0.5	3.3 / 6.8%

pred·i·cate

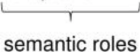


noun **GRAMMAR**

/ˈpredəkət/

the part of a sentence or clause containing a verb and stating something about the subject (e.g., **went home** in **John went home**).

"predicate adjective"

- The relationship between a noun phrase and a predicate
 - John threw a ball to Mary.
 - A ball was thrown to Mary by John.
 - ...
 - John: the thrower
 - a ball: entity thrown
 - Mary: recipient

semantic roles
- Semantically normalize syntactic variation

- Automatically identify semantic roles
- Example
 - [Sender John] [Predicate shipped] [Thing shipped a package]
[Source from Michigan] [Destination to California].
- Benefits downstream processes
 - Question answering (Pizzato and Molla, 2008)
 - Information extraction (Banko and Etzioni, 2008)
 - Machine translation (Liu and Gildea, 2010)

Semantic Roles

Underlying relations between
predicates and their arguments.

actor	destination	patient	theme
agent	experiencer	predicate	theme
asset	extent	product	patient
attribute	instrument	recipient	time
beneficiary	location	source	topic
cause	material	stimulus	proposition

Named entity recognition

An example

Company

c

Location

l

Date

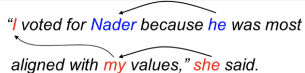
d

Based in the **UK** x, **Datavid** x is a data engineering firm that provides services to organizations around the world since **2018** x.

Coreference Resolution

Overview

Coreference resolution is the task of finding all expressions that refer to the same entity in a text. It is an important step for a lot of higher level NLP tasks that involve natural language understanding such as document summarization, question answering, and information extraction.



"I voted for Nader because he was most aligned with my values," she said.

The diagram shows three curved arrows indicating coreference relations: one from 'I' to 'she', one from 'he' to 'Nader', and one from 'my' to 'she'.